UNITÀ 3 · APPLICAZIONI JAVA PER LA COMUNICAZIONE IN RETE

La chat con i socket TCP

Un server, tanti client, un thread per ciascuno e un piccolo protocollo di testo: costruiamo da zero la stessa idea che sta dietro a ogni app di messaggi.

PER COMINCIARE

Con quale app scrivi di più ai compagni?

SONDAGGIO

- **A** WhatsApp
- **B** Telegram
- **C** Instagram
- **D** Discord
- **E** Un'altra

UNA STORIA DEL 1971

Il primo messaggio tra due computer in rete

1971, BBN, Cambridge (USA). **Ray Tomlinson** manda un messaggio tra due computer PDP-10 collegati ad ARPANET, uno accanto all'altro. Per separare utente e computer sceglie il simbolo **@**.

1988, Università di Oulu (Finlandia). **Jarkko Oikarinen** crea **IRC**: canali, nickname e comandi di testo come `NICK` e `PRIVMSG`.

> **I messaggi di prova erano del tutto dimenticabili, e quindi li ho dimenticati.**
>
> **Ray Tomlinson**, ingegnere, nel suo racconto "The First Network Email"

RIPASSO VELOCE

Socket TCP: che cosa ci dà e che cosa no

ServerSocket

Ascolta su una porta. `accept()` si blocca finché arriva un client e restituisce un nuovo **Socket** per parlare con lui.

Socket

Un capo della connessione: indirizzo IP + porta. Da qui prendi un InputStream e un OutputStream.

Niente messaggi

TCP è un **flusso di byte** affidabile e ordinato. Dove finisce un messaggio lo decide il nostro **protocollo**: noi useremo una riga.

> Nella lezione su Claude Code un agente correggeva `ServerChat.java` e `GestoreClient.java`. Oggi quei file li scriviamo noi, e capiamo da dove veniva quel bug.

PUZZLE

Metti in ordine la vita di una connessione

1. Il server crea `new ServerSocket(5000)`
2. `accept()` restituisce un Socket nuovo per quel client
3. Il client crea `new Socket(host, 5000)` : parte l'handshake TCP
4. Il client chiude: sul server `readLine()` restituisce `null`
5. Il server chiama `accept()` e si blocca
6. Client e server si scambiano righe di testo

IL CODICE · 1

ServerEco: ripete in maiuscolo quello che riceve

```
try (ServerSocket server = new ServerSocket(5000)) {
    System.out.println("In ascolto sulla porta 5000");
    while (true) {
        try (Socket s = server.accept();
             var in = new BufferedReader(new InputStreamReader(s.getInputStream(), UTF_8));
             var out = new PrintWriter(s.getOutputStream(), true, UTF_8)) {
            System.out.println("Connesso: " + s.getRemoteSocketAddress());
            String riga;
            while ((riga = in.readLine()) != null)       // null = il client ha chiuso
                out.println("ECO " + riga.toUpperCase());
        }
    }
}
```

Il `true` di PrintWriter è l'**autoflush**: senza, la risposta resta nel buffer e il client aspetta per sempre.

AL COMPUTER · 15 minuti

Il server che fa eco

1. Nella cartella **u3-chat** crea **ServerEco.java** e avvialo: `java ServerEco.java`
2. In un secondo terminale collegati con `curl telnet://127.0.0.1:5000`
3. Scrivi qualche riga e premi Invio; chiudi con Ctrl+C.
4. Ora apri **due** client insieme e scrivi da entrambi.
5. Annota in **eco.txt** che cosa succede al secondo client.

Controllo: scrivendo `ciao` ricevi `ECO CIAO` e il server stampa `Connesso: /127.0.0.1:` seguito da una porta alta, diversa per ogni client.

QUIZ A TRABOCCHETTO

ServerEco sta servendo un client. Se ne collega un secondo: che cosa succede?

QUIZ

A Riceve subito "Connection refused"

B Si connette, ma non riceve risposte finché il primo non esce

C I due client vengono serviti a turno, una riga ciascuno

D Il server va in errore e si chiude

L'ARCHITETTURA

Un thread per client, una lista condivisa

ServerChat

Ciclo infinito: `accept()`, poi crea un thread e torna subito ad ascoltare.

GestoreClient anna

GestoreClient bruno

GestoreClient carla

clienti

Un `Set` thread-safe con tutti i gestori: per inoltrare un messaggio si scorre il Set e si scrive a ognuno.

IL CODICE · 2

ServerChat.java

```
public class ServerChat {
    static final Set<GestoreClient> clienti = ConcurrentHashMap.newKeySet();

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(5000)) {
            System.out.println("Chat in ascolto sulla porta 5000");
            while (true) {
                Socket s = server.accept();                 // aspetta un client
                new Thread(new GestoreClient(s)).start();   // un thread per client
            }
        }
    }

    static void inoltra(String riga, GestoreClient mittente) {
        for (GestoreClient c : clienti)
            if (c != mittente) c.invia(riga);
    }
}
```

IL CODICE · 3

GestoreClient: il cuore del metodo `run()`

```
try (s; var in = new BufferedReader(new InputStreamReader(s.getInputStream(), UTF_8))) {
    out = new PrintWriter(s.getOutputStream(), true, UTF_8);
    String riga = in.readLine();                                  // 1a riga: NICK nome
    if (riga == null || !riga.matches("NICK \\w{1,16}")) { invia("ERR serve NICK nome"); return; }
    nick = riga.substring(5);
    ServerChat.clienti.add(this);
    invia("OK benvenuto " + nick);
    ServerChat.inoltra("INFO " + nick + " entra nella chat", this);
    while ((riga = in.readLine()) != null && !riga.equals("QUIT")) {
        if (riga.startsWith("MSG ")) ServerChat.inoltra("MSG " + nick + " " + riga.substring(4), this);
        else invia("ERR comando sconosciuto");
    }
} catch (IOException e) {
    System.out.println("Connessione persa: " + e.getMessage());
} finally {                                                       // sempre, anche dopo un errore
    if (ServerChat.clienti.remove(this)) ServerChat.inoltra("INFO " + nick + " esce dalla chat", this);
}
```

IL PROTOCOLLO

Una riga = un comando. Le regole le decidiamo noi

Client → server	Significato
`NICK anna`	Mi presento (prima riga)
`MSG ciao a tutti`	Messaggio per tutti
`QUIT`	Esco

Server → client	Significato
`OK benvenuto anna`	Comando accettato
`ERR serve NICK nome`	Errore, con il motivo
`MSG anna ciao`	Messaggio di anna
`INFO bruno entra…`	Avviso del server

Un protocollo è un **contratto**: formato, ordine dei messaggi, risposte agli errori. Scriverlo prima del codice permette a due gruppi di fare client e server separatamente.

AL COMPUTER · 30 minuti

La chat multi-client

1. Crea **ServerChat.java** e **GestoreClient.java** nella stessa cartella.
2. Compila e avvia: `javac *.java` e poi `java ServerChat`
3. Apri tre client con `curl telnet://127.0.0.1:5000`; nel primo scrivi `NICK anna`, negli altri `NICK bruno` e `NICK carla`.
4. Da anna scrivi `MSG ciao`, poi chiudi la finestra di bruno senza QUIT.
5. Collegati al server di un compagno con il suo IP.

Controllo: bruno e carla ricevono `MSG anna ciao`; anna no. Quando chiudi bruno, gli altri ricevono `INFO bruno esce dalla chat` e il server continua a funzionare.

METTITI ALLA PROVA

Sul server, `in.readLine()` restituisce `null`. Che cosa è successo?

QUIZ

A Il client ha chiuso la connessione

B Il client ha mandato una riga vuota

C Per 30 secondi non è arrivato nulla

D Il messaggio era più lungo del buffer

AL COMPUTER · 30 minuti

Il client Java con due thread

Un client deve fare due cose insieme: leggere la tastiera e ascoltare il server. Crea **ClientChat.java**:

1. Apri `new Socket(host, 5000)` con i suoi stream in UTF-8.
2. Avvia con `Thread.ofVirtual().start(...)` un thread che stampa ogni riga del server.
3. Nel main leggi la tastiera e invia ogni riga; esci dopo `QUIT`.
4. Prova con due ClientChat e un client curl.

Controllo: i messaggi degli altri compaiono anche mentre stai scrivendo; dopo `QUIT` il programma termina e gli altri vedono `INFO ... esce dalla chat`.

PUZZLE

Abbina ogni pezzo di codice al suo compito

server.accept()	---	Un Set che più thread possono modificare insieme
new PrintWriter(os, true)	---	Scrive righe e le invia subito (autoflush)
readLine() == null	---	L'altro capo ha chiuso la connessione
ConcurrentHashMap.newKeySet()	---	Toglie il client dalla lista anche dopo un errore
finally	---	Aspetta un client e restituisce il suo Socket

SICUREZZA

Chi può leggere la nostra chat?

- Il testo viaggia **in chiaro**: con Wireshark chiunque sulla stessa rete può leggerlo.
- Rimedio di base: **TLS**, con `SSLServerSocket` e `SSLSocket`. Cifra il canale tra client e server.
- Ma il server, che decifra, legge comunque tutto.



IERI E OGGI

IRC non è morto, e i thread sono diventati leggeri





Discord, WhatsApp, i giochi online: tutti tengono una **connessione sempre aperta** verso un server, come la nostra chat. Nel browser, quella connessione si chiama **WebSocket**: è la prossima lezione.

ALL'ESAME

Progettare un servizio client-server

1. Disegna l'**architettura**: un server, N client, porta, protocollo di trasporto (TCP o UDP, e perché).
2. Scrivi il **protocollo**: tabella di comandi e risposte, compresi gli errori.
3. Disegna un **diagramma di sequenza**: due client, il server, un messaggio inoltrato.
4. Spiega la **concorrenza**: un thread per client e la lista condivisa thread-safe.

Domanda classica

"Perché TCP e non UDP per una chat?" Perché serve che ogni messaggio arrivi, una volta sola e nell'ordine giusto. Per voce e video in diretta invece si preferisce UDP: un pacchetto in ritardo è inutile.

BIGLIETTO D'USCITA

Completa prima di uscire

Il metodo che aspetta un nuovo client si chiama _____.

TCP non trasporta messaggi ma un flusso di _____: i confini li decide il protocollo.

Se l'altro capo chiude, `readLine()` restituisce _____.

Per servire più client insieme si usa un _____ per ogni client.

Per togliere il client dalla lista anche dopo un errore si usa il blocco _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- ServerSocket ascolta, accept() restituisce un Socket per ogni client
- Un server a thread singolo serve un client alla volta
- Un thread per client e un Set thread-safe per inoltrare
- Protocollo di testo: una riga = un comando, con OK ed ERR
- null = connessione chiusa; finally per non lasciare fantasmi
- In chiaro chiunque legge: TLS, o meglio end-to-end

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.